

CSE 3105 – Microprocessors & Embedded Systems
Final Exam Preparation Q&A (8086 Assembly)

1. Data Transfer Instructions

Q1. Write an assembly program to move the value 1234H into AX and then copy AX into BX.

```
.model small
.stack 100h
.code
main proc
    mov ax, 1234h
    mov bx, ax
    mov ah, 4Ch
    int 21h
main endp
end main
```

Answer:

- AX = 1234H, BX = 1234H
- Demonstrates MOV instruction and register-to-register transfer.

Q2. Explain MOV AX, [NUM1].

- Loads 16-bit value from memory location NUM1 into AX.
- Memory content is not changed.

2. Arithmetic Instructions

Q3. Add two 16-bit numbers NUM1=1000H and NUM2=2000H and store the result in SUM.

```
.model small
.stack 100h
.data
NUM1 dw 1000h
NUM2 dw 2000h
SUM dw ?
.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, NUM1
    add ax, NUM2
    mov SUM, ax

    mov ah, 4Ch
    int 21h
main endp
end main
```

Answer: SUM = 3000H

Q4. Subtract two numbers NUM1=1234H, NUM2=432H, store result in RESULT.

```
.model small
.stack 100h
.data
    NUM1    dw 1234h
    NUM2    dw 0432h
    RESULT  dw ?

.code
main proc
    mov ax, @data
    mov ds, ax    ; initialize data segment

    mov ax, NUM1    ; AX = 1234h
    sub ax, NUM2    ; AX = AX - NUM2 = 1234h - 0432h = 0FEDh
    mov RESULT, ax  ; store result

    mov ah, 4Ch
    int 21h        ; return to DOS
main endp
end main
```

Answer: RESULT = 0FEDH

Q5. Multiply two 8-bit numbers NUM1=5, NUM2=3 and store 16-bit result in RESULT.

```
.model small
.stack 100h
.data
    NUM1    db 5
    NUM2    db 3
    RESULT  dw ?

.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, NUM1    ; AL = 5
    mov bl, NUM2    ; BL = 3
    mul bl          ; AX = AL * BL = 15 (0Fh)
    mov RESULT, ax  ; store 16-bit result

    mov ah, 4Ch
    int 21h
main endp
end main
```

Answer: RESULT = 15 (0F H)

Q6. Divide AX by BL and store quotient in AL, remainder in AH.

```
.model small
.stack 100h
.data
    QUOTIENT db ?
    REMAINDER db ?

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, 15    ; AX = 15
    mov bl, 4     ; BL = 4
    div bl        ; AL = AX/BL = 3, AH = AX mod BL = 3

    mov QUOTIENT, al ; store quotient
    mov REMAINDER, ah ; store remainder

    mov ah, 4Ch
    int 21h
main endp
end main
Answer: AL=3, AH=3
```

3. Logic Instructions

Q7. Perform AND, OR, XOR operations.

```
.model small
.stack 100h
.data
    RESULT1 db ?
    RESULT2 db ?
    RESULT3 db ?

.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, 0F0h
    and al, 0FFh
    mov RESULT1, al

    mov al, 0F0h
    or al, 0Fh
    mov RESULT2, al
```

```

mov al, 0F0h
xor al, 0FFh
mov RESULT3, al

```

```

    mov ah, 4Ch
    int 21h
main endp
end main

```

Answer: Demonstrates masking, setting, clearing bits.

4. Branching Instructions

Q8. Explain:

- JZ LABEL → Jump if Zero Flag set
- JNZ LABEL → Jump if Zero Flag clear
- JC LABEL → Jump if Carry Flag set
- LOOP LABEL → Decrement CX, jump if CX ≠ 0

Example Program:

```

MOV AX, 0
MOV CX, 3
MOV BX, 1
LOOP1:
    ADD AX, BX
    INC BX
    LOOP LOOP1

```

Answer:

- Final AX = 6 (1+2+3)
 - ADD executed 3 times
-

5. Input/Output Instructions (INT 21H)

Q9. Read a character and display it back:

```

.model small
.stack 100h
.data
.code
main proc
    mov ax, @data
    mov ds, ax

    ; Read a character from keyboard
    mov ah, 1    ; DOS function 01h: read char
    int 21h     ; AL = input character

    ; Display the same character
    mov ah, 2    ; DOS function 02h: display char
    mov dl, al   ; DL = character to print
    int 21h

```

```
    mov ah, 4Ch
    int 21h    ; Exit to DOS
main endp
end main
Answer: Echoes keyboard input.
```

Q10. Display string "HELLO":

```
.model small
.stack 100h

.data
msg db 'HELLO $' ; string to display (must end with $)

.code
main proc
    ; initialize data segment
    mov ax, @data
    mov ds, ax

    ; display string
    mov ah, 9    ; DOS function 09h - print string
    lea dx, msg  ; load offset of msg into DX
    int 21h     ; call DOS interrupt

    ; exit program
    mov ah, 4Ch
    int 21h
main endp
end main
```

Answer: Prints HELLO on screen.

6. Loops

Q11. Find sum of numbers from 1 to 5 using LOOP instruction:

```
.model small
.stack 100h

.data
SUM dw 0    ; variable to store the sum

.code
main proc
    mov ax, @data
    mov ds, ax

    mov cx, 5 ; loop counter from 5 down to 1
    mov ax, 0 ; AX will hold the running sum
    mov bx, 1 ; BX = current number to add
```

```

sum_loop:
    add ax, bx          ; add current number to AX
    inc bx              ; move to next number
    loop sum_loop       ; decrement CX and loop if CX ≠ 0

    mov SUM, ax         ; store result in SUM

; exit program
mov ah, 4Ch
int 21h
main endp
end main

```

7. Stack Instructions

Q12. Demonstrate PUSH and POP:

```

.model small
.stack 100h
.data
    RESULT1 dw ?
    RESULT2 dw ?

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, 1234h      ; AX = 1234h
    mov bx, 5678h      ; BX = 5678h

    push ax            ; push 1234h
    push bx            ; push 5678h

    pop cx             ; CX = 5678h (last pushed)
    pop dx             ; DX = 1234h (first pushed)

    mov RESULT1, cx    ; store CX = 5678h
    mov RESULT2, dx    ; store DX = 1234h

    mov ah, 4Ch
    int 21h            ; exit to DOS
main endp
end main

```

Answer: Values reversed using stack.

8. Flag Instructions

Q13. Explain flags affected by arithmetic:

- ZF (Zero Flag): Set if result=0
- CF (Carry Flag): Set if carry in addition or borrow in subtraction

- SF (Sign Flag): Set if result negative
- OF (Overflow Flag): Set if signed overflow

Q14. Write conditional jump example:

```
.model small
.stack 100h
.data
    msg1 db 'Numbers are EQUAL$'
    msg2 db 'Numbers are NOT EQUAL$'

.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, 5      ; AL = 5
    sub al, 5      ; AL = AL - 5 → result = 0, ZF = 1
    jz EQUAL      ; Jump if Zero flag = 1

NOT_EQUAL:        ; if result ≠ 0
    mov ah, 9
    lea dx, msg2   ; print "NOT EQUAL"
    int 21h
    jmp EXIT

EQUAL:            ; if result = 0
    mov ah, 9
    lea dx, msg1   ; print "EQUAL"
    int 21h

EXIT:
    mov ah, 4Ch
    int 21h        ; Exit to DOS
main endp
end main
```

Answer: Jumps to EQUAL because AL-ZF=0

9. Memory Addressing Modes

Q15. Explain:

- **Immediate:** MOV AX, 5
 - **Direct:** MOV AX, NUM1
 - **Register:** MOV AX, BX
 - **Register Indirect:** MOV AX, [BX]
 - **Indexed:** MOV AX, [SI+5]
 - **Based-Indexed:** MOV AX, [BX+SI+4]
-

10. String Operations

Q16. Copy 5 bytes from SRC to DEST using REP MOVSB:

```
.model small
.stack 100h
.data
    SRC    db 'HELLO'    ; source string (5 bytes)
    DEST   db 5 dup(?)    ; destination (5 empty bytes)

.code
main proc
    mov ax, @data
    mov ds, ax            ; DS → data segment

    lea si, SRC           ; SI = offset of SRC
    lea di, DEST          ; DI = offset of DEST
    mov cx, 5             ; number of bytes to copy
    cld                   ; clear direction flag (left-to-right)
    rep movsb            ; copy CX bytes from DS:SI → ES:DI

    mov ah, 4Ch
    int 21h               ; exit to DOS
main endp
end main
```

Answer: 5 bytes copied from SRC to DEST.

Tips

Know DOS interrupts:

- 01h → Read char, 02h → Write char, 09h → Print string, 4Ch → Exit

Extra 8086 Assembly Exam Preparation Q&A

1. Arithmetic & Data Handling

Q1. Write a program to calculate the square of an 8-bit number stored in memory (NUM1) and store the 16-bit result in RESULT.

```
.model small
.stack 100h
.data
    NUM1 db 07h
    RESULT dw ?
.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, NUM1
    mov bl, al
    mul bl            ; AL * BL → AX
    mov RESULT, ax
```

```

    mov ah, 4Ch
    int 21h
main endp
end main

```

Answer: If NUM1=07H, RESULT=0031H (7×7=49).

Q2. Explain the difference between MUL BL and IMUL BL.

- MUL BL → Unsigned multiplication (AL × BL → AX).
- IMUL BL → Signed multiplication (AL × BL → AX). Handles negative numbers correctly.

Q5. Write a program using conditional jump to check if a number in AL is even or odd.

```

.model small
.stack 100h
.data
    msg db 'The number is: $'

.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, 7        ; number to check
    test al, 1       ; check LSB (0 = even, 1 = odd)
    jz EVEN         ; jump if zero (even)

    ; Odd case
    mov bl, 'O'
    jmp PRINT

EVEN:
    mov bl, 'E'

PRINT:
    ; Display message
    mov ah, 9
    lea dx, msg
    int 21h

    ; Display result character
    mov ah, 2
    mov dl, bl
    int 21h

    mov ah, 4Ch
    int 21h        ; exit to DOS
main endp
end main

```

Answer: BL = 'O' since 7 is odd.

Q6. Explain CMP AL, BL followed by JG NEXT.

- CMP subtracts BL from AL (AL - BL) without storing result.
 - JG NEXT jumps if AL > BL (signed comparison).
-

3. Stack & Subroutines

Q7. Write a program to exchange two numbers using stack.

```
.data
a db 5      ; define variable a = 5
b db 9      ; define variable b = 9

.code
main proc
    mov ax, @data    ; initialize data segment
    mov ds, ax

    mov al, a        ; load value of a into AL
    mov bl, b        ; load value of b into BL

    push ax          ; push AL (a) onto stack
    push bx          ; push BL (b) onto stack

    pop al           ; pop top of stack into AL (gets value of b)
    pop bl           ; pop next value into BL (gets value of a)

    mov a, al        ; store AL (b) into variable a
    mov b, bl        ; store BL (a) into variable b

    mov ah, 4Ch      ; terminate program
    int 21h
main endp
end main
```

Q8. Explain difference between CALL and RET.

- CALL label → pushes IP (next instruction) on stack, jumps to subroutine.
 - RET → pops IP from stack, returns to calling point.
-

4. Input/Output (INT 21h extended)

Q10. Write a program to read two characters and display them with a space in between.

```
mov ah, 1
int 21h
mov bl, al

mov ah, 1
int 21h
mov bh, al
```

```
mov ah, 2
mov dl, bl
int 21h
```

```
mov dl, ' '
int 21h
```

```
mov dl, bh
int 21h
```

Q11. Write a program to display a new line and carriage return.

```
mov ah, 2
mov dl, 10 ; carriage return
int 21h
mov dl, 13 ; line feed
int 21h
```

Answer: Moves cursor to next line.

5. Flags & Comparisons

Q12. Write a program to compare two numbers and display the greater one.

```
.model small
.stack 100h
.data
    msg db 10,13, 'Largest Number is: $'

    num1 db 5 ; first number
    num2 db 9 ; second number

.code
main proc
    mov ax, @data
    mov ds, ax

    ; load numbers
    mov bl, num1
    mov cl, num2

    ; compare
    cmp bl, cl
    jg level1 ; if num1 > num2, go level1
    jmp level2 ; else, go level2

level1:
    mov ah, 9
    lea dx, msg
    int 21h
```

```

mov ah, 2
mov dl, bl
add dl, 30h    ; convert to ASCII
int 21h
jmp exit

```

level2:

```

mov ah, 9
lea dx, msg
int 21h

```

```

mov ah, 2
mov dl, cl
add dl, 30h    ; convert to ASCII
int 21h

```

exit:

```

mov ah, 4Ch
int 21h

```

main endp

end main

Q13. Explain effect of CMP on flags.

- ZF = 1 if operands equal.
- CF = 1 if borrow occurs (AL < BL).
- SF = set if result negative.
- OF = set if signed overflow.

Q15. Explain difference between MOVSB and STOSB.

- MOVSB: Copy byte from DS:SI → ES:DI.
- STOSB: Store AL into ES:DI.

Q16. Explain the use of Direction Flag (DF).

- DF=0 (CLD) → SI and DI increment after string ops (left-to-right).
- DF=1 (STD) → SI and DI decrement after string ops (right-to-left).

7. Mixed/Applied Problems

Q17. Write a program to display all digits 0–9 on screen.

```

.model small
.stack 100h

.code
main proc
    mov ax, @data
    mov ds, ax

```

```
mov cx, 10    ; loop counter for 10 digits (0–9)
mov dl, 0     ; starting digit = 0
```

```
display_loop:
  add dl, 30h  ; convert digit to ASCII ('0' = 30h)
  mov ah, 2    ; DOS function 02h - display character
  int 21h      ; display the character
  sub dl, 30h  ; restore digit value for next iteration
  inc dl       ; move to next digit
  loop display_loop ; decrement CX and repeat if CX ≠ 0
```

```
; exit program
mov ah, 4Ch
int 21h
main endp
end main
```

Answer: Prints 0123456789.

Q20. Explain the difference between INC AX and ADD AX,1.

- Both increase AX by 1.
 - INC does **not** affect Carry Flag.
 - ADD affects Carry Flag.
-